

Introduction to neural networks

Recurrent Networks

Juan Irving Vásquez
jivg.org

Centro de Innovación y Desarrollo Tecnológico en Cómputo
Instituto Politécnico Nacional

May 28, 2026©



Contents

- 1 Motivation
- 2 Recurrent network
- 3 Training & Challenges
- 4 Extensions: LSTM and GRU
- 5 Application to NLP

Roadmap

- 1 Motivation
- 2 Recurrent network
- 3 Training & Challenges
- 4 Extensions: LSTM and GRU
- 5 Application to NLP

Sequential Data

- Order matters. Current output depends on history.
- Examples:
 - **Time Series:** Stock prices, weather data.
 - **NLP:** Translation, Sentiment Analysis.
 - **Audio:** Speech recognition.

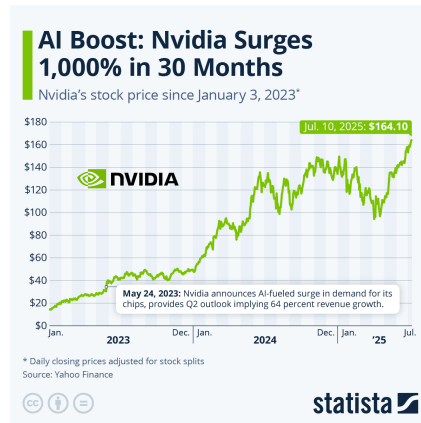


Figure: Nvidia Stock prices ^a

Motivation

Using feed forward networks in sequential data

- Feed forward networks (FFN), e.g. the multilayer perceptron, have fixed input size.
- Nevertheless, they can be used as follows:
 - Size larger than input (the input is filled)
 - Size equals input (rarely occurs)
 - Size is shorter than input (only a section is analyzed)

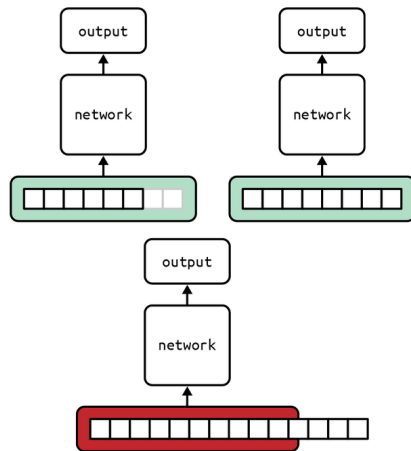
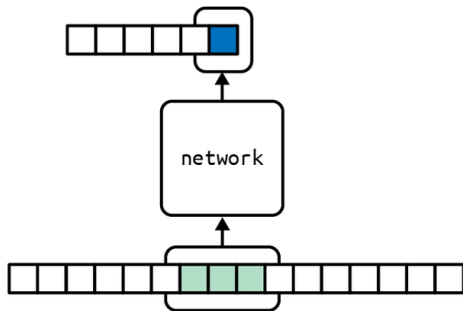


Figure: Different size inputs problem.

Seq2seq with Neural N-Grams

Example of using FFN in sequences

Then the woman , after grabbing her umbrella , went to the bank to deposit her cash .
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
RB DT NN , IN VBG PRP\$ NN , VBD TO DT NN TO VB PRP\$ NN .



Feedforward vs. Recurrent

- Feedforward Neural Network (FNN)
 - Information flows in one direction:
Input $\rightarrow$ Hidden $\rightarrow$ Output.
- Recurrent Neural Network (RNN).
Contains a **loop**. The output of a neuron is fed back into itself as input for the next time step.
 - Maintains an internal state (memory).
 - Uses **Shared Weights** across all time steps.

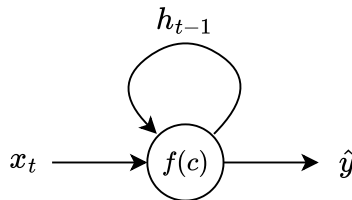


Figure: Simple recurrent neural node

Roadmap

- 1 Motivation
- 2 Recurrent network
- 3 Training & Challenges
- 4 Extensions: LSTM and GRU
- 5 Application to NLP

Recurrent neural network

At time step t , the hidden state h_t is updated using the input x_t and the previous hidden state h_{t-1} .

$$\mathbf{h}_t = f(W_h \mathbf{h}_{t-1} + W_x X_t + b) \quad (1)$$

where:

- $\mathbf{h}_t$: New hidden state.
- $\mathbf{h}_{t-1}$: Old hidden state (memory from previous step).
- X_t : Current input vector.
- W_h, W_x : Weight matrices (Trainable parameters).
- $f(c)$: Activation function.

Recurrent neural network

The prediction is based on the hidden state.

- For simplicity:

$$\hat{y} = \mathbf{h}_T \quad (2)$$

where T is the length of the input.

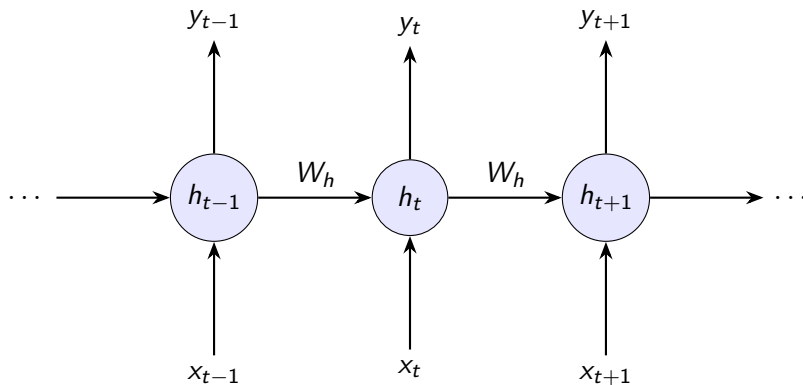
- in general:

$$\hat{y} = \mathbf{h}_T W_y \quad (3)$$

where W_y is a trainable matrix.

Unrolling through time

To understand RNNs, we visualize them as multiple copies of the same network, each passing a message to a successor.



Roadmap

- 1 Motivation
- 2 Recurrent network
- 3 Training & Challenges**
- 4 Extensions: LSTM and GRU
- 5 Application to NLP

Backpropagation Through Time (BPTT)

- Training is similar to standard Backpropagation, but gradients flow backward through time.
- We compare the predicted sequence to the target sequence to compute Loss (L).
- We compute $\frac{\partial L}{\partial W}$ by summing gradients at each time step.

Backproagation implementation

- In the output layer you have error at time T (last time),
- We can generalize previous error term as δ_t ,
- Term error for hidden layer node t ,

$$\delta_t = w_h \delta_{t+1} f'(c_t), \quad (4)$$

where c is the linear combination

- The increment is computed by time,

$$\Delta w_{x|t} = \eta \delta_t x_i \quad (5)$$

$$\Delta w_{h|t} = \eta \delta_t h_i \quad (6)$$

$$\Delta b_t = \eta \delta_t (1) \quad (7)$$

The Vanishing Gradient Problem

The Problem:

- Since the same matrix W is multiplied repeatedly (chain rule) during backpropagation...
- If weights are small (< 1), gradients shrink exponentially → **Vanishing Gradient**.
- If weights are large (> 1), gradients grow exponentially → **Exploding Gradient**.

Consequence

The network effectively stops learning from early inputs in long sequences.
It suffers from **Short-Term Memory**.

Example:

*"I grew up in **France**... [long paragraph] ... I speak fluent **French**."*

An RNN with vanishing gradients might forget the context "France" by the time it needs to predict "French".

Roadmap

- 1 Motivation
- 2 Recurrent network
- 3 Training & Challenges
- 4 Extensions: LSTM and GRU**
- 5 Application to NLP

Solution: Long Short-Term Memory (LSTM)

Introduced by Hochreiter & Schmidhuber en 1997¹.

- Introduces a **Cell State** (C_t) acting as an information highway.
- Uses **Gates** to regulate information flow:
 - ① **Forget Gate:** What to remove from memory?
 - ② **Input Gate:** What new info to store?
 - ③ **Output Gate:** What to show as output?

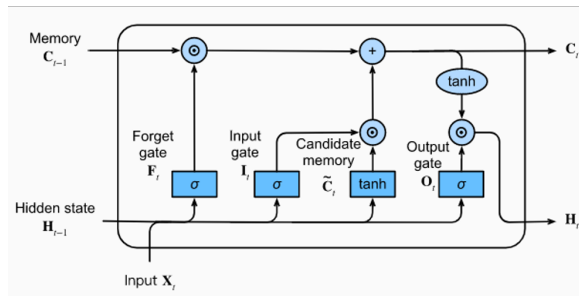


Figure: LSTM neural network

¹Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8), 1735-1780.

LSTM (Cell state)

Cell state computation:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (8)$$

where,

- f_t : forget gate. How much of the past cell state should be forgotten,

$$f_t = \sigma(W_{fx}x_t + W_{fh}h_{t-1} + b_f)$$

- i_t : The input gate. It decides which new information from the current input will be allowed to update the cell state,

$$i_t = \sigma(W_{ix}x_t + W_{ih}h_{t-1} + b_i)$$

- $\tilde{C}_t$: The candidate cell state,

$$\tilde{C}_t = \tanh(W_{cx}x_t + W_{ch}h_{t-1} + b_c)$$

- $\odot$: Hadamard product (element-wise multiplication).

LSTM (Hidden state)

Hidden state computation:

$$h_t = o_t \odot \tanh(C_t) \quad (9)$$

where o_t is the output gate:

$$o_t = \sigma(W_{ox}x_t + W_{oh}h_{t-1} + b_o) \quad (10)$$

Alternative: Gated Recurrent Unit (GRU)

Introduced by Cho et al., en 2014².

- A simplified version of LSTM.
- Merges the Cell State and Hidden State.
- Combines Forget and Input gates into a single **Update Gate**.
- **Benefit:** Fewer parameters, faster to train, often performs just as well.

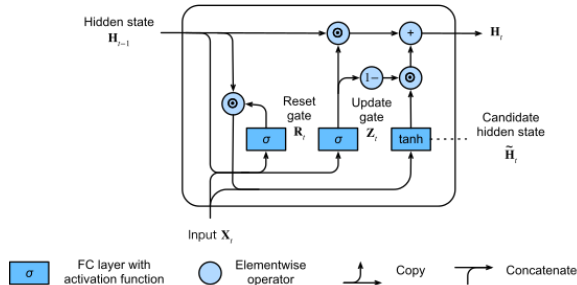


Figure: GRU neural network

²Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y., 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078.

Roadmap

- 1 Motivation
- 2 Recurrent network
- 3 Training & Challenges
- 4 Extensions: LSTM and GRU
- 5 Application to NLP**

A functional definition

Using machine learning and large datasets to give computers the ability **NOT** to understand language, but to ingest a piece of language as input and return something useful.

Predicting the useful output:

- *"What's the topic of this text?"* → **Text classification**
- *"Does this text contain abuse?"* → **Content filtering**
- *"Does this text sound positive or negative?"* → **Sentiment analysis**
- *"What should be the next word in this incomplete sentence?"* → **Language modeling**
- *"How would you say this in German?"* → **Translation**
- *"How would you summarize this article in one paragraph?"* → **Summarization**

Text Processing Models

There are two main kinds of text-processing models:

- **Sequence models:** Care about word order. They use word-level tokenization.
- **Bag-of-words models:** Treat input words as a set, discarding their original order. They use N-gram tokenization.

Sequence-to-Sequence (Seq2Seq) tasks

- Are tasks where the inputs and outputs are sequences.
- Example (Machine translation):
 - Input: *J'aime le thé.* (French)
 - Output: *I love tea.* (English)
- **Special Tokens:** They are additional to the sentence.
 - <bos>: Beginning of sentence.
 - <eos>: End of sentence.

- Deep learning models are differentiable functions that can only process numeric tensors: **they cannot take raw text as input.**
- **Vectorization:** The process of transforming text into numeric tensors.

Original Text: ("The cat sat on the mat.")

- 1 **Standardization:**
("the cat sat on the mat")
- 2 **Tokenization:**
("the", "cat", "sat", "on", "the", "mat")
- 3 **Indexing:**
(3, 26, 65, 9, 3, 133)
- 4 **One-hot encoding:**
(Vector Representation).

Tokenization

There are different strategies for splitting text:

- **Word-level tokenization:** Tokens are space-separated (or punctuation-separated) substrings.
- **N-gram tokenization:** Tokens are groups of N consecutive words (e.g., "the cat" is a 2-gram or bigram).
- **Character-level tokenization:** Each character is its own token.

Large Language Models (LLMs), such as GPT-3 and GPT-4, utilize a process called tokenization. Tokenization involves breaking down text into smaller units, known as tokens, which the model can process and understand. These tokens can range from individual characters to entire words or even larger chunks, depending on the model. For GPT-3 and GPT-4, a Byte Pair Encoding (BPE) tokenizer is used. BPE is a subword tokenization technique that allows the model to dynamically build a vocabulary during training, efficiently representing common words and word fragments. Although the core tokenization process remains similar across different versions of these models, the specific implementation can vary based on the model's architecture and training objectives.

Definition

Word N-grams are groups of N (or fewer) consecutive words that can be extracted from a sentence.

Example: "the cat sat on the mat."

- **Set of 2-grams:** {"the", "the cat", "cat", "cat sat", "sat", "sat on", "on", "on the", "the mat", "mat"}.
- **Set of 3-grams:** {"the", "the cat", "cat", "cat sat", "the cat sat", "sat", ...}.

Vocabulary Indexing

- Once text is split into tokens, they must be encoded numerically.
- In practice, an index of all terms in the training data (the "vocabulary") is built, and a unique integer is assigned to each entry.

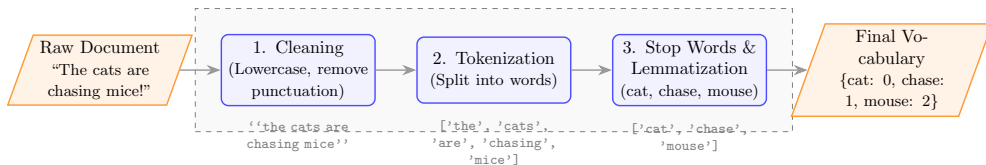
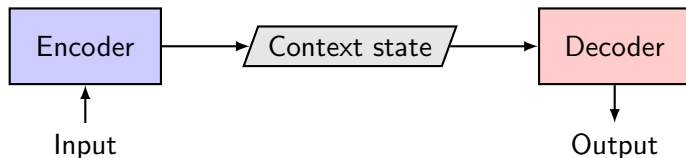


Figure: Vocabulary creation

Encoder-decoder architecture

- **Problem Domain:** Seq2seq problems.
- **Variable Lengths:** Generally, input and output sequences have different lengths.
- **The architecture** for this problem:
 - **Encoder:** Converts a variable-length sequence into a *fixed-length* output state.
 - **Context state:** Acts as a bottleneck or compressed representation.
 - **Decoder:** Converts the fixed-length state back into a variable-length output.



The Encoder: Mathematical Formulation

- The input sequence is represented as vectors, $x_1, \dots, x_T$, (after tokenization and embedding).

$$X = \begin{bmatrix} [x_{1,1} & x_{1,2} & \dots & x_{1,d}] \\ \vdots \\ [x_{n,1} & x_{n,2} & \dots & x_{n,d}] \end{bmatrix} \quad (11)$$

- A RNN generates a new hidden state h_t based on the previous state h_{t-1} and current input x_t :

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, x_t) \quad (12)$$

Concretely (using tanh activation):

$$\mathbf{h}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x x_t + b) \quad (13)$$

The Context Vector

- The **Context Variable (or Vector)** c must encode the information of the *entire* input sequence.
- It is defined by a mapping function m :

$$c = m(\mathbf{h}_1, \dots, \mathbf{h}_T) \quad (14)$$

- In the simplest case, it is just the last hidden state of the encoder:

$$c = \mathbf{h}_T \quad (15)$$

The Decoder

- The decoder generates the output sequence $\hat{y}_1, \dots, \hat{y}_{T'}$.
- Inputs to decoder:
 - 1 Context variable: c
 - 2 Previous output token: $\hat{y}_{t'-1}$
 - 3 Previous decoder hidden state: $s_{t'-1}$
- Decoder state update

$$s_{t'} = g(s_{t'-1}, \hat{y}_{t'-1}, c) \quad (16)$$

Output Probability

The conditional distribution of the next token is calculated via Softmax:

$$P(\hat{y}_{t'} | \hat{y}_{t'-1}, \dots, \hat{y}_1, c) = \text{SoftMax}(s_{t'}) \quad (17)$$

Training the Seq2Seq Model

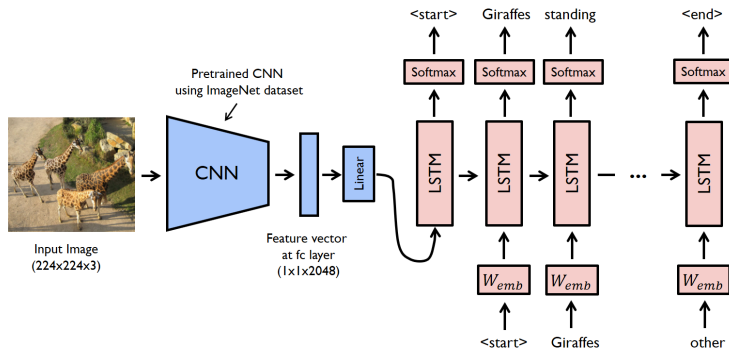
- **Joint Training:** The encoder and decoder are trained together.
- **Loss Function:** Cross-entropy loss is used for optimization:

$$\max_{\theta} \frac{1}{N} \sum_{n=1}^N \log p_{\theta}(\hat{y}^{(n)} | x^{(n)}) \quad (18)$$

- **Teacher Forcing:**
 - A technique to address slow convergence and instability.
 - The ground truth (original output sequence) is used as input to the decoder during training, rather than the model's own previous prediction.

Great advances!

- FCN, CNN, RNN, ResNet were having great succesful



- But they were limited by the supervised approach
- RNN struggled to parse longer chunks of text

- **RNNs** are designed for sequential data where history matters.
- They share weights across time steps (W_h, W_x).
- Training is done via **Backpropagation Through Time (BPTT)**.
- **Vanishing Gradients** make standard RNNs hard to train on long sequences.
- **LSTMs and GRUs** decrease this using gating mechanisms to preserve long-term dependencies.

- Du, K.L. and Swamy, M.N., 2013. Neural networks and statistical learning. Springer Science & Business Media.
- Zhang, A., Lipton, Z.C., Li, M. and Smola, A.J., 2023. Dive into deep learning. Cambridge University Press.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. Deep learning. MIT press, 2016.
- NYU, Aprendizaje profundo,
<https://atcold.github.io/pytorch-Deep-Learning/es/week07/07-3/>